

stats-cli-py

AI Agent 技能架构文档

构建方式 | 最佳实践 | AI Agent 集成

版本: 1.2.1

日期: 2026-06-15

命令: 33 | 测试: 794 | 覆盖率: 92%

目录

- | | |
|------------------|-------------------|
| 1. 技能概述 | stats-cli-py 是什么? |
| 2. 架构设计 | 三层架构 |
| 3. AI Agent 调用方式 | JSON 分发器模式 |
| 4. 渐进式披露 | 两层设计 |
| 5. 最佳实践评分 | 8 维度评估 |
| 6. 决策树 | 统计方法路由 |
| 7. 自发现机制 | discover 命令 |
| 8. 改进记录 | 我们做了哪些改进 |

1. 技能概述

stats-cli-py 是一个纯 Python 统计分析 CLI/库，专为制造业和质量工程设计。它提供 33 个统计命令，涵盖描述性统计、假设检验、SPC 控制图、过程能力分析、DOE 实验设计、MSA 测量系统分析、可靠性分析、多变量分析、时间序列等领域。

核心设计原则：

- 所有 I/O 均为 JSON 格式：通过 stdin/stdout 或文件交互
- 纯 Python 实现：基于 scipy + statsmodels，无 R 依赖
- AI Agent 原生：自描述 API，内置 discover 命令
- 渐进式披露：简单入口，按需深入

命令分类 (33 个命令)

分类	命令
基础统计	descriptive, normality, outlier
假设检验	ttest, anova, nonparametric, homogeneity, equivalence, power
SPC 控制图	control_chart, capability, trend
回归分析	regression, correlation
实验设计	doe (全因子, 分数因子, 田口, 响应曲面)
测量系统	gage_rr (交叉, 嵌套, 属性)
可靠性	reliability (威布尔, Kaplan-Meier)
多变量	multivariate (PCA, 聚类, 判别)
时间序列	timeseries (指数平滑, ARIMA, 分解)
数据处理	explore, clean, transform, discover
工作流	workflow, workflow_template, check_assumptions, recommend
报告导出	report, export_excel, export_pdf, run

2. 架构设计

技能采用三层架构：

第1层：路由层（main.py）

解析 JSON 输入，通过 `_route()` 函数路由命令（延迟导入），处理异常，将输出包装在标准信封中。共 374 行代码。

第2层：执行层（stats_engine/*.py）

28 个独立模块，每个模块暴露一个公开函数，接受 `**params` 参数并返回 dict。所有计算使用 `scipy.stats`、`numpy` 和 `statsmodels`，无自定义统计实现。

第3层：工具层（utils/*.py）

4 个工具模块：`data_loader`（文件加载，自动编码检测）、`data_cleaner`（NaN/Inf 处理）、`validators`（输入验证）、`output`（JSON 信封，数值四舍五入）。

数据流

```
输入 JSON -> main.py handler() -> _route(command, params)
    -> stats_engine/module.function(**params)
    -> utils/output.success(data) 或 error(msg)
    -> JSON 标准输出
```

3. AI Agent 调用方式

三种调用模式

```
# 模式 1: stdin 管道
echo '{"command":"descriptive", "params":{"values":[1,2,3]}}' | python main.py

# 模式 2: 文件参数
python main.py input.json

# 模式 3: Python 直接调用
from main import handler
result = handler({"command": "descriptive", "params": {"values": [1,2,3]}})
```

标准输出信封

```
成功: {"status":"success", "version":"1.2.1", "timestamp":"...", "data":{...}}
失败: {"status":"error", "error_type":"...", "message":"...", "suggestion":"..."}
```

Agent 调用流程 (7 步)

1. Agent 收到用户模糊请求 ("帮我分析数据")
2. Agent 调用 discover -> 获取 33 个命令摘要
3. Agent 根据 description/category 选择合适命令
4. Agent 调用 discover {command_name:"xxx"} -> 获取参数 schema
5. Agent 用 example 作为模板构造 JSON 请求
6. Agent 调用目标命令 -> 获取结果
7. Agent 用 output_fields 解析结果, 向用户解释

4. 渐进式披露

技能实现了两层渐进式披露：

第1层：SKILL.md 静态文档（4 级）

第1级：Quick Start 决策树（5 个顶层分支）

- > 第2级：4 个详细决策树（比较/关系/预测/质量控制）
- > 第3级：5 个场景化工作流（完整 JSON 示例）
- > 第4级：33 个命令完整参考（所有参数）

第2层：discover 运行时动态发现（2 级）

第1级：discover（无参数）-> 轻量摘要（description + category）

- > 第2级：discover {command_name:"xxx"} -> 完整 schema（params + output_fields）

为什么需要两层？

静态 SKILL.md 在技能激活时加载一次，提供决策树和场景覆盖常见用例。discover 命令在运行时查询，用于 SKILL.md 未覆盖的边缘场景，或当 Agent 需要精确参数 schema 时。

5. 最佳实践评分

评估维度	评分	关键优势
触发条件	Strong	双语触发词，覆盖面广
渐进式披露	M-Strong	两层设计，决策树入口
决策引导	Strong	4 个决策树，recommend 命令
错误处理	Moderate	7 种错误类型，suggestion 字段
示例驱动	Strong	5 个场景，33 个命令示例
自描述性	Strong	运行时 discover，结构化 params
组合能力	M-Strong	workflow，9 个模板，auto_check
参数文档	Strong	结构化 params，含 required/type

核心设计亮点

- discover 命令：运行时自描述，Agent 无需读文档即可了解能力
- 决策树：编码正确的统计方法论（先检验正态性，再选参数/非参数）
- recommend 命令：元引导 - Agent 可以直接问工具该用什么方法
- workflow_template：9 个预制模板覆盖常见分析场景
- 结构化参数：discover 中每个参数含 required/type/desc

6. 决策树

比较分析决策树

用户想比较数据

```

|
+-- 只有 2 组?
|   +-- 正态? -> ttest (two_sample)
|   +-- 非正态? -> nonparametric mann_whitney
|       +-- 也考虑: transform boxcox -> 重新检验
|
+-- 3+ 组?
|   +-- 正态? -> anova one_way
|   +-- 非正态? -> nonparametric kruskal_wallis
|
+-- 配对数据?
    +-- 正态? -> ttest paired
    +-- 非正态? -> nonparametric wilcoxon
  
```

质量控制决策树

用户想做质量控制

```

|
+-- 监控稳定性? -> control_chart (imr/xbar/r)
+-- 检查能力? -> capability (需要 USL/LSL)
+-- 测量系统? -> gage_rr (交叉/嵌套)
+-- 可靠性? -> reliability (威布尔/Kaplan-Meier)
+-- 检测趋势? -> trend (cusum/ewma/runs)
  
```


7. 自发现机制（discover 命令）

discover 命令是运行时自描述机制，让 AI Agent 无需读取 SKILL.md 即可了解可用命令。

三种查询模式

模式 1：列出所有命令（轻量）

```
{"command": "discover"}
```

模式 2：按类别筛选

```
{"command": "discover", "params": {"category": "spc"}}
```

模式 3：单命令详情（完整 schema）

```
{"command": "discover", "params": {"command_name": "capability"}}
```

结构化参数（v1.2.1）

每个命令现在暴露结构化参数元数据：

```
"params": [  
  {"name": "values", "required": true, "type": "array", "desc": "过程数据"},  
  {"name": "usl", "required": true, "type": "number", "desc": "规格上限"},  
  {"name": "lsl", "required": true, "type": "number", "desc": "规格下限"},  
  {"name": "target", "required": false, "type": "number", "desc": "目标值"}  
]
```

这消除了猜测 - Agent 准确知道哪些参数是必需的，哪些是可选的。

8. 改进记录

v1.2.1 改动 (2026-06-15)

SKILL.md 改进:

- + 添加 Intent-to-Command 快速查找表 (19 个命令)
- + 在文档顶部推荐 Agent 优先使用 discover 命令
- + 添加负面触发词 (不支持: 流数据、文本、图像、非数值)
- + 添加 Top 5 命令输出示例
- + 添加错误恢复工作流表
- + 文档化 workflow auto_check 失败语义
- + 文档化 run 命令沙箱限制
- + 在决策树中添加 Box-Cox 交叉链接
- + 将所有占位 [...] 替换为可运行数据

代码改进:

- + discover.py 添加结构化 params (33 个命令)
- + 修复 fpdf2 弃用警告 (report.py)
- + 修复 Matplotlib 弃用 (charts.py)
- + 异常改为 logging 记录而非吞没

打包:

- + pyproject.toml: 版本 1.2.1, 添加 fpdf2 依赖, 修复 build-backend
- + Zip: 排除 TODO.md、CLAUDE.md、测试文件、构建产物
- + 最终 zip: 47 个文件, 310 KB